

Fondamenti di Informatica

A.A. 2017-2018

Prova intermedia del 6 giugno 2018 – Testo 1

NOME _____ COGNOME _____

Durante l'esame non si possono usare appunti né altro materiale.

Esercizio 1

Si dica se ciascuna delle seguenti affermazioni relative ai *puntatori* in C++ è corretta:

Sì No

- | | | |
|--------------------------|--------------------------|--|
| ☐ | ☐ | I valori che una variabile puntatore può assumere sono di norma numeri reali tra 0 e 1. |
| ☐ | ☐ | De-referenziare un puntatore nullo non produce mai errori in fase di esecuzione. |
| ☐ | ☐ | Il nome di un array corrisponde a un puntatore al suo primo elemento. |
| ☐ | ☐ | Un puntatore rappresenta l'indirizzo di memoria di una variabile o di una funzione. |
| ☐ | ☐ | È possibile moltiplicare due puntatori. |
| ☐ | ☐ | Quando si dichiara un puntatore viene sempre allocata un'area di memoria per l'oggetto puntato. |
| ☐ | ☐ | Un puntatore può puntare ad un altro puntatore: in tal caso si ha un puntatore a puntatore. |
| ☐ | ☐ | L'espressione &x restituisce l'indirizzo della locazione di memoria in cui risiede la variabile x. |
| ☐ | ☐ | Gli elementi di un array non possono essere puntatori. |
| ☐ | ☐ | Un puntatore il cui valore è 0 o NULL è detto puntatore nullo e non punta ad alcun oggetto. |

Esercizio 2

Si dica che cosa stampa a video il seguente programma C++, ovvero quali nodi compongono la lista *l* dopo aver eseguito le operazioni sotto riportate:

```
#include "List.h"
int main() {
    List<int> l;
    l.insertFront(30);
    l.insertBack(18);
    l.insertBack(20);
    l.insertFront(26);
    l.insertAfter(l.find(18), 28);
    l.removeBack();
    l.print();
    return 0;
}
```

Soluzione:

Esercizio 3

Completare le seguenti affermazioni:

1. Una pila è una lista nella quale si inseriscono o si rimuovono elementi solo dalla _____.
2. Secondo il principio di indipendenza dalla rappresentazione, due implementazioni corrette di una stessa specifica di un tipo di dato astratto sono _____.
3. Alle _____, sebbene siano funzioni esterne, è concesso di accedere ai membri privati di una classe.
4. Per accedere all'oggetto puntato da una variabile puntatore si applica l'operatore _____.

Esercizio 4

Identificare e correggere gli errori nella seguente serie di istruzioni:

```
1      typename <template T>
2      T calcolaMin(T a[], int n) {
3          int min = a[0];
4          for (int i = 0; i < n; i++)
5              if (a[i] < min)
6                  min = a[i];
7          return <T>min;
8      }
```

Soluzione:

I seguenti esercizi (Esercizio 5 ed Esercizio 6) dovranno essere svolti al calcolatore e consegnati su AulaWeb. Accertarsi che la consegna sia confermata dal sistema!

Esercizio 5

Allo scopo di contribuire allo sviluppo del sistema informatico di un istituto bancario, si sviluppi in linguaggio C++ la classe *ContoCorrente* avente i seguenti attributi: il numero del conto corrente (un numero intero), il codice CAB della filiale presso la quale ha sede il conto (un numero intero), un codice identificativo dell'intestatario (un numero intero) e il saldo in Euro del conto corrente (un numero reale). Si implementino, inoltre, i seguenti metodi:

- Il costruttore di default che inizializzi a zero tutti gli attributi, usando la lista di inizializzazione.
- Un costruttore con parametri che riceva come parametri il valore del numero di conto corrente, del codice CAB, del codice identificativo dell'intestatario e del saldo e inizializzi i corrispondenti attributi della classe usando la lista di inizializzazione. Per semplicità, si assuma che i valori ricevuti dai parametri siano sempre validi.
- Il distruttore.
- I selettori (un metodo per ciascun attributo).
- I modificatori (un metodo per ciascun attributo). Per semplicità, si assuma che i valori ricevuti dai parametri dei modificatori siano sempre validi.
- Gli operatori di inserimento e di estrazione.

Si sviluppi quindi la funzione *main* che operi come segue:

- Apra in lettura il file *Conti.txt*. Tale file contiene nella sua prima riga il numero di righe *n* di cui il file si compone e nelle righe successive, per ciascuna di esse, le informazioni relative ad un conto corrente nel seguente ordine: numero di conto corrente, codice CAB, identificativo dell'intestatario e saldo.
- Legga la prima riga del file *Conti.txt* e allochi dinamicamente un array *cc* di *n* oggetti di tipo *ContoCorrente*.
- Sfruttando l'operatore di estrazione, sopra definito per la classe *ContoCorrente*, legga le righe successive del file *Conti.txt* e assegni agli elementi dell'array *cc* le informazioni contenute in ciascuna riga del file. Per semplicità, si supponga che il file non sia vuoto e che i dati in esso contenuti siano validi.
- Scandisca l'array *cc* e stampi a video il numero di conti corrente il cui saldo è superiore a 50.000 Euro.
- Deallochi l'array *cc*, chiuda il file e termini.

Esercizio 6

1. Si scriva la funzione C++ *AbsDiff* che riceva come parametri due numeri reali e restituisca come valore di ritorno la differenza in valore assoluto tra i due valori passati come parametri (un numero reale).
2. Si scriva la funzione C++ *Prod* che riceva come parametri due numeri reali e restituisca come valore di ritorno il prodotto dei due valori passati come parametri (un numero reale).
3. Si scriva la funzione C++ *Calcola* che riceva come parametri due array *a* e *b* di numeri reali, la loro dimensione *n* e il puntatore *pf* a una funzione che riceva come parametri due numeri reali e restituisca come valore di ritorno un numero reale. La funzione scandirà gli array *a* e *b* elemento per elemento, ad ogni iterazione chiamerà la funzione puntata da *pf* passandole come parametri la corrispondente coppia di elementi di *a* e *b* e sommerà tra loro i valori di ritorno restituiti dalla funzione chiamata. La funzione ritornerà infine il valore di tale somma (un numero reale) oppure 0 nel caso in cui il puntatore *pf* sia nullo.
4. Si scriva infine un programma C++ che operi come segue: chieda all'utente di inserire da tastiera due array *x* e *y* di 5 numeri reali, chieda all'utente se desidera calcolare la norma 1 della differenza dei due array o il loro prodotto scalare, chiami la funzione *Calcola* di cui al punto 3, passando come parametro per *pf* la funzione *AbsDiff* nel primo caso e la funzione *Prod* nel secondo caso, stampi a video il risultato e termini.

Soluzione

Esercizio 1 – Soluzione (1.5 punti, 0.15 per ogni risposta esatta)

No, No, Sì, Sì, No, No, Sì, Sì, No, Sì.

Esercizio 2 – Soluzione (1.0 punto)

Dopo le operazioni elencate, i nodi della lista contengono i seguenti valori (che verranno stampati a video): 26 30 18 28.

Esercizio 3 – Soluzione (1.0 punto, 0.25 per ogni risposta esatta)

Testa, (osservabilmente) indistinguibili, funzioni amiche, di de-referenziazione (*).

Esercizio 4 – Soluzione (1.5 punti, 0.5 per ogni risposta esatta)

Riga 1: typename <template T> -> **template** <typename T>.

Riga 3: errore di tipo int min = a[0]; -> **T** min = a[0];

Riga 7: rimuovere <T> -> **return min;**

Esercizio 5 – Soluzione (4.0 punti, 0.8 – 0.2 – 0.2 – 0.1 – 0.15 – 0.15 – 0.15 – 0.15 - 0.15 – 0.15 – 0.15 – 0.15 – 0.5 - 0.5 – 0.5)

```
#include <iostream>
#include <fstream>
using namespace std;

class ContoCorrente {
public:
    ContoCorrente();
    ContoCorrente(int numero, int CAB,
                  int intest, double saldo);
    ~ContoCorrente();
    int GetNumero() const;
    int GetCAB() const;
    int GetIntestatario() const;
    double GetSaldo() const;
    void SetNumero(int numero);
    void SetCAB(int CAB);
    void SetIntestatario(int intest);
    void SetSaldo(double saldo);
    friend istream& operator>> (istream& is, C
                               ContoCorrente& cc);
    friend ostream& operator<< (ostream& os, const
                               ContoCorrente& cc);

private:
    int _numero;
    int _CAB;
    int _intest;
    double _saldo; };

ContoCorrente::ContoCorrente()
: _numero(0), _CAB(0), _intest(0), _saldo(0.0) {}

ContoCorrente::ContoCorrente(int numero, int CAB,
                              int intest, double saldo)
: _numero(numero), _CAB(CAB), _intest(intest),
  _saldo(saldo) {}

ContoCorrente::~ContoCorrente() {}

int ContoCorrente::GetNumero() const {
    return _numero; }

int ContoCorrente::GetCAB() const {
    return _CAB; }

int ContoCorrente::GetIntestatario() const {
    return _intest; }

double ContoCorrente::GetSaldo() const {
    return _saldo; }

void ContoCorrente::SetNumero(int numero) {
    _numero = numero; }

void ContoCorrente::SetCAB(int CAB) {
    _CAB = CAB; }

void ContoCorrente::SetIntestatario(int intest) {
    _intest = intest; }

void ContoCorrente::SetSaldo(double saldo) {
    _saldo = saldo; }

istream& operator>> (istream& is,
                    ContoCorrente& cc) {
    is >> cc._numero >> cc._CAB
      >> cc._intest >> cc._saldo;
    return is; }

ostream& operator << (ostream& os,
                    const ContoCorrente& cc) {
    os << "Numero: " << cc._numero << endl;
    os << "CAB: " << cc._CAB << endl;
    os << "Intestatario: " << cc._intest << endl;
    os << "Saldo: " << cc._saldo << endl;
    return os; }

int main() {
    fstream fin;
    fin.open("Conti.txt", ios::in);
    if (!fin) return -1;
    int n = 0; fin >> n;
    ContoCorrente* cc = new ContoCorrente[n];
    for (int i = 0; i < n; i++)
        fin >> cc[i];
    int contatore = 0;
    for (int j = 0; j < n; j++)
        if (cc[j].GetSaldo() > 50000)
            contatore++;
    cout << "Conti con piu' di 50000 Euro: "
         << contatore << endl;
    delete[] cc;
    fin.close();
    return 0; }
```

Esercizio 6 – Soluzione (2.0 punti, 0.25 – 0.25 – 0.75 – 0.75)

```
#include <iostream>
#include <cmath>
using namespace std;

double AbsDiff(double x, double y) {
    return abs(x - y); }

double Prod(double x, double y) {
    return x*y; }

double Calcola(double a[], double b[], int n,
               double(*pf)(double, double)) {
    if (pf == NULL) return 0;
    double somma = 0.0;
    for (int i = 0; i < n; i++)
        somma += pf(a[i], b[i]);
    return somma; }

int main() {
    const int dim = 2;
    double x[dim]; double y[dim];
    cout << "Inserire due array di " << dim << " reali: ";
    for (int i = 0; i < dim; i++)
        cin >> x[i] >> y[i];
    int scelta = 0;
    cout << "Calcolo normal (0) o prodotto scalare (1)? ";
    cin >> scelta;
    if (scelta == 0)
        cout << Calcola(x, y, dim, AbsDiff) << endl;
    else
        cout << Calcola(x, y, dim, Prod) << endl;
    return 0; }
```